

Phase 17 — 最小必测矩阵（小白逐步版）

本文件与 docs/157_phase17_staging_validation_playbook.md 对齐，用更细步骤说明「最少要测什么、怎么测」。

不要把 App Secret、refresh token、用户密码、完整 access token 发给任何人或贴进聊天软件；只放在你自己的电脑/服务器的配置里。

1. 三个名词（先搞懂再动手）

名词	白话
Staging（预发/测试环境）	不是客户正式在用的那一套；用来试错。可以是本机 Docker、公司测试服务器、云上一台小机器。
Phase 0 / A / B / C	docs/157 里的四段验收顺序：先基线（0）→ 可选轮换演练（A）→ Zalo 换票（B）→ Meta 换票（C）。
Smoke（冒烟测试）	用脚本快速打一遍接口，确认服务「能响、没挂」。本仓库命令：npm run smoke:webhooks。

2. 最小必测矩阵（你是不是每个聊天软件都要测？）

结论：不是。

「每个都要做一遍」的是 Phase 0 的 webhook 冒烟（一条命令可覆盖七路，有验签时再跳过部分）。

Phase B / C（进程内换票）只需要在 Zalo 和你实际上线的 Meta 线路（WhatsApp / Messenger）上做。

通道	Phase 0 冒烟	Phase A 轮换演练	Phase B/C 换票真实测试
Website	建议测	若生产用则做	不必（无换票逻辑）
Telegram	建议测	若生产用则做	不必
WhatsApp	建议测	若生产用则做	若开换票开关则必测（C）
Messenger	建议测	若生产用则做	若开换票且上线 Messenger 则测（C）
Line	建议测	若生产用则做	不必
Zalo	建议测	若生产用则做	若开换票开关则必测（B）

最小组合建议（小白默认）：

1. 所有人先做 → **Phase 0**（整站冒烟）。
 2. 准备对生产打开 `CHATFLOW_INPROCESS_TOKEN_REFRESH` 时 → 加 **Phase B（Zalo）**、**Phase C（你用的 Meta 线）**。
 3. **Phase A** → 不着急可以后面补；想练运维再加。
-

3. 开始之前：你要准备的东西

- ☐ 已安装 **Node.js**（与项目一致，建议 22.x）、**npm**。
 - ☐ 已克隆本仓库到本机，能进入项目根目录（里面有 `package.json`）。
 - ☐（可选）已安装 **Docker Desktop**，用于本机起服务（见 `docs/158`）。
 - ☐ 若要测 **云端 staging**：有一个 **https 开头的公网地址** 指向你的 ChatFlow（或内网穿透），且防火墙放行端口。
 - ☐ 一个**记事本**只给自己看：记录每次测试日期、结果、**X-Request-Id**（不要抄 token）。
-

4. Phase 0 — 基线（换票功能先关掉）

目的：确认服务正常、七路 webhook 能跑通（或按规则跳过）。

4.1 确认环境变量

1. 打开你的 staging 配置（`.env` 或云平台环境变量面板）。
2. 确认 `CHATFLOW_INPROCESS_TOKEN_REFRESH` **没有**设为 1、true、yes（**不设最省事**）。
3. 保存后**重启** ChatFlow 进程（改 env 一般要重启才生效）。

4.2 起服务（二选一）

方式 A — 本机 Docker 一键（有 Docker 时）

1. 打开终端，进入项目根目录。
2. 执行：

```
npm run staging:docker-smoke
```

 - 会：起容器 → 等健康检查 → 跑 smoke → 关掉容器。
 - 若 **3030 端口被占用**：先设端口再跑，例如 PowerShell：

```
$env:STAGING_HOST_PORT='3031'; npm run staging:docker-smoke
```
3. 终端里应看到 `[smoke] all passed`。

方式 B — 本机直接跑 Node

1. npm run build
2. npm run start (另开终端保持运行)
3. 再开终端: npm run smoke:webhooks
 - 默认访问 http://127.0.0.1:3030。

方式 C — 测云端地址

1. 确认云上服务已部署且能访问。
2. 在本机项目根目录执行:
SMOKE_BASE_URL=https://你的域名 npm run smoke:webhooks
3. 若配置了 **POST 验签**, 部分通道会失败, 需按 docs/152 设置 SMOKE_SKIP_CHANNELS (例如跳过 website)。这是**正常现象**, 在记事本写「跳过哪些通道」。

4.3 记录结果

- ☐ Phase 0: 通过 / 部分跳过 (写明跳过原因)
 - ☐ 抄一个响应头里的 X-Request-Id 备查 (不含密钥)
-

5. Phase A — 轮换演练 (可选, 练运维)

目的: 学会「在平台改 token → 重启服务 → 业务仍正常」, 与 docs/152 一致。

1. 只选 **一条** 通道 (例如 WhatsApp)。
 2. 登录 **Meta 开发者后台** (或对应平台), 在**测试应用**里轮换/更新 token (不要发到聊天)。
 3. 把**新 token**写进 staging 的 **环境变量**, 保存。
 4. **重启** ChatFlow。
 5. 再跑一次 **Phase 0** 的 smoke 或手动发一条真实消息, 确认仍 **200**、业务正常。
 6. 另一条通道可以**改天再做**, 不必一天全测完。
-

6. Phase B — Zalo 进程内换票 (仅当你要在生产开换票时必做)

目的: 故意让 access token 无效, 验证是否会 refresh 并重试, 且日志/JSON 里有约定证据。

6.1 只在 staging 配置

在 **staging** 环境变量中设置 (**不要**提交到 git) :

- CHATFLOW_INPROCESS_TOKEN_REFRESH=1
- ZALO_REFRESH_TOKEN (有效 refresh)
- ZALO_APP_ID、ZALO_APP_SECRET

- 平时就有的：ZALO_ACCESS_TOKEN、ZALO_OA_ID 等（见 docs/154）

保存并 **重启** 服务。

6.2 制造「坏 access token」（推荐小白做法）

1. **复制**当前正确的 ZALO_ACCESS_TOKEN 到记事本备份（仅本地）。
2. 在 staging 里把 ZALO_ACCESS_TOKEN **临时改成** invalid_for_test 之类明显错误值。
3. 保存并 **重启** 服务。

6.3 发一条 Zalo webhook

1. 打开 docs/129，找到 **Zalo** 的 **POST /webhooks/zalo** 示例（curl + JSON）。
2. 把 URL 里的主机改成你的 staging 地址（本机则用 http://127.0.0.1:3030）。
3. 在终端执行 curl（或 Postman 同等请求）。
4. 期望：HTTP **200**（本产品设计为对调用方仍返回 200，具体见 docs/154）；在返回 JSON 的 debug_steps 或服务器日志里查找 zalo_real_token_refresh_retry。

6.4 收尾（Rollback）

1. 把 ZALO_ACCESS_TOKEN **改回**记事本里的正确值。
2. 将 CHATFLOW_INPROCESS_TOKEN_REFRESH **关掉**（删掉或设 0），若暂时不用换票。
3. 重启服务，再跑一次 **Phase 0** 确认恢复常态。

7. Phase C — Meta（WhatsApp / Messenger）换票（仅当你要在生产开换票时必须做）

目的：在 Graph 返回 **401** 或 **400** 且 **error code 190** 时，验证 fb_exchange_token 路径是否触发。

7.1 只在 staging 配置

- CHATFLOW_INPROCESS_TOKEN_REFRESH=1
- META_APP_ID
- META_APP_SECRET 或 WHATSAPP_APP_SECRET 或 MESSENGER_APP_SECRET（其一，与 docs/156 一致）
- 对应通道：WHATSAPP_ACCESS_TOKEN + Phone Number ID，和/或 Messenger 的 page token + page id

保存并 **重启**。

7.2 分两条线（不必同一天全做）

线	什么时候必须测
WhatsApp Cloud	生产要用 WhatsApp 真发 → 建议必测
Messenger	生产不用 Messenger → 可跳过 ；要用 → 再测

7.3 制造 Graph 认为 token 无效

常用思路（在 **Meta 测试应用**里操作，不要动生产页）：

- 在开发者后台 **撤销**当前测试 token，或
- 临时把环境变量里的 access token **改成错误字符串**（与 Zalo 同理），

然后对 **对应通道** 发 docs/129 里的 **POST** /webhooks/whatsapp 或 /webhooks/messenger。

7.4 看什么算成功

在 debug_steps 或日志中查找：

- WhatsApp: whatsapp_real_meta_token_exchange_retry
- Messenger: messenger_real_meta_token_exchange_retry

7.5 Rollback

关掉 CHATFLOW_INPROCESS_TOKEN_REFRESH，按 docs/152 恢复合法 token，重启，再 smoke。

8. 可复制：最小证据清单

- [] Phase 0: 完成（日期：_____） 跳过通道：_____
- [] Phase A: 完成 / 跳过 通道：_____ 日期：_____
- [] Phase B（Zalo）：观察到 zalo_real_token_refresh_retry: 是 / 否 日期：_____
- [] Phase C（WhatsApp）：观察到 whatsapp_real_meta_token_exchange_retry: 是 / 否 日期：_____
- [] Phase C（Messenger）：观察到 messenger_real_meta_token_exchange_retry: 是 / 否 日期：_____
- [] Rollback 已做: 是 / 否

9. 如何导出为 PDF（任选一种）

方法 A — VS Code / Cursor（最简单）

1. 用编辑器打开本文件 docs/160_phase17_minimal_test_matrix_beginner.md。
2. 打开 Markdown 预览（预览图标或命令面板搜 “Markdown: Open Preview”）。

3. 在预览窗口右键 → **Print** → 打印机选 **Microsoft Print to PDF** / **另存为 PDF** → 保存。

方法 B — 浏览器

- 1. 把本 .md 拖到支持 Markdown 的查看器，或用 GitHub / Gitea 网页打开该文件。
- 2. **打印** → **另存为 PDF**。

方法 C — 仓库自带脚本（Windows + 本机已装 Microsoft Edge，推荐）

在项目根目录执行：

```
npm run docs:pdf:160
```

会生成：

- docs/160_phase17_minimal_test_matrix_beginner_print.html（可双击用浏览器打开再打印）
- docs/160_phase17_minimal_test_matrix_beginner.pdf

依赖：npx marked（首次自动下载）；脚本见 scripts/build-doc160-pdf.mjs。

说明：该脚本在 **Windows** 上通过 **Microsoft Edge** 无头生成 PDF。**macOS / Linux** 请用 **方法 A 或 B**：打开仓库里的 docs/160_phase17_minimal_test_matrix_beginner_print.html（与 PDF 一并提交），用浏览器 **打印** → **存储为 PDF**；或自行用 npx marked 从本 .md 生成 HTML 后打印。

方法 D — md-to-pdf（可选，可能需下载 Chromium，较慢）

```
npx --yes md-to-pdf docs/160_phase17_minimal_test_matrix_beginner.md
```

方法 E — Pandoc（已安装时）

```
pandoc docs/160_phase17_minimal_test_matrix_beginner.md -o docs/160_phase17_minimal_test_matrix_beginner.pdf
```

10. 参考文档（按顺序）

文档	内容
docs/157	总 playbook（精简版）
docs/158	Docker 本机 staging
docs/152	Token 轮换与 smoke 跳过验签
docs/154	Zalo 换票实现说明
docs/156	Meta 换票说明

文档	内容
docs/129	七路 curl 与示例 body

文档版本：与 ChatFlow Pro docs/157 对齐；导出 PDF 前请确认已拉取最新 main。